UPSSA
UNIVERSITÉ PRIVÉE DE LA SANTÉ ET DES SCIENCES D'AGADIR
الجامعة الخاصة للصحة و العلوم لأكادير

Module : POO en Java
Corrigé Travaux pratiques 1

Exercice 1 : Bonjour

Énoncé

Écrivez un programme qui affiche « Bonjour, je m'appelle ... » avec votre prénom, puis votre âge sur une deuxième ligne.

Solution (fichier Exercice01.java)

```
public class Exercice01 {
    public static void main(String[] args) {
        String prenom = "Sara";
        int age = 22;

        System.out.println("Bonjour, je m'appelle " + prenom + ".");
        System.out.println("J'ai " + age + " ans.");
    }
}
```

Exemple d'exécution

```
Bonjour, je m'appelle Sara.
J'ai 22 ans.
```

Commentaires

À retenir

- ✓ System.out.println affiche le texte puis passe à la ligne ; System.out.print affiche sans retour à la ligne.
- ✓ L'opérateur + entre un texte et une variable fait une concaténation : l'entier age est converti automatiquement en texte.
- ✓ String (avec un S majuscule) est une classe ; int est un type primitif. On range le prénom dans un String et l'âge dans un int.

Erreurs fréquentes et conseils

- ! Le nom du fichier doit être identique au nom de la classe publique : Exercice01.java pour public class Exercice01.
- ! Oublier le point-virgule en fin d'instruction, ou fermer une chaîne sans guillemet, sont les deux premières erreurs de compilation que vous rencontrerez.
- ! Remplacez « Sara » et 22 par vos propres informations.

Exercice 2 : Calculs simples

Énoncé

Déclarez deux entiers a = 12 et b = 5. Affichez leur somme, leur différence, leur produit, leur quotient entier et le reste de la division (%).

Solution (fichier Exercice02.java)

```
public class Exercice02 {
    public static void main(String[] args) {
        int a = 12;
        int b = 5;

        System.out.println("Somme       : " + (a + b));
        System.out.println("Différence  : " + (a - b));
        System.out.println("Produit     : " + (a * b));
        System.out.println("Quotient    : " + (a / b));   // division entière
        System.out.println("Reste       : " + (a % b));
    }
}
```

Exemple d'exécution

```
Somme       : 17
Différence  : 7
Produit     : 60
Quotient    : 2
Reste       : 2
```

Commentaires

À retenir

✓ Opérateurs arithmétiques : +, -, *, / et % (modulo, c'est-à-dire le reste de la division entière).

✓ Quand les deux opérandes sont des int, / réalise une division entière : 12 / 5 donne 2 (la partie décimale est supprimée, pas arrondie).

✓ Vérification : 12 = 5 × 2 + 2, donc quotient = 2 et reste = 2.

Erreurs fréquentes et conseils

! Les parenthèses sont indispensables dans (a + b) à l'intérieur d'un println : sans elles, "Somme : " + a + b afficherait 125 (concaténation de 12 et de 5).

! Beaucoup d'étudiants s'attendent à obtenir 2.4 pour 12 / 5. Pour un résultat décimal, il faut convertir un des opérandes : (double) a / b donne 2.4.

Exercice 3 : Lecture au clavier

Énoncé

Demandez à l'utilisateur son nom et son année de naissance (avec Scanner), puis affichez : « Bonjour Sara, vous avez environ 22 ans. »

Solution (fichier Exercice03.java)

```
import java.time.Year;
import java.util.Scanner;

public class Exercice03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Votre nom : ");
        String nom = sc.nextLine();

        System.out.print("Votre année de naissance : ");
        int anneeNaissance = sc.nextInt();

        int anneeCourante = Year.now().getValue();
        int age = anneeCourante - anneeNaissance;

        System.out.println("Bonjour " + nom + ", vous avez environ " + age + " ans.");
        sc.close();
    }
}
```

Exemple d'exécution

```
Votre nom : Sara
Votre année de naissance : 2004
Bonjour Sara, vous avez environ 22 ans.
```

Commentaires

À retenir

✓ Pour lire au clavier : import java.util.Scanner; puis new Scanner(System.in). nextLine() lit une ligne complète, nextInt() lit un entier.

✓ L'âge est calculé par différence entre l'année courante (Year.now().getValue()) et l'année de naissance. Il est « environ » correct car on ignore le mois et le jour de naissance.

✓ Pensez à fermer le Scanner avec close() quand vous n'en avez plus besoin.

Erreurs fréquentes et conseils

! Piège classique : si vous appelez nextInt() puis nextLine(), ce dernier renvoie une chaîne vide, car nextInt() laisse le retour à la ligne dans le tampon. Ici le nom est lu en premier, donc tout va bien. Si l'ordre était inversé, il faudrait ajouter un sc.nextLine() pour consommer ce retour à la ligne.

! Si l'utilisateur tape du texte alors qu'un entier est attendu, le programme s'arrête avec une InputMismatchException.

! Le résultat dépend de l'année en cours : avec 2004, on obtient 22 ans en 2026.

Exercice 4 : Rectangle

Énoncé

Lisez la longueur et la largeur d'un rectangle (nombres réels), puis affichez son périmètre et son aire.

Solution (fichier Exercice04.java)

```
import java.util.Locale;
import java.util.Scanner;

public class Exercice04 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        sc.useLocale(Locale.US);   // le séparateur décimal sera le point

        System.out.print("Longueur : ");
        double longueur = sc.nextDouble();
        System.out.print("Largeur  : ");
        double largeur = sc.nextDouble();

        double perimetre = 2 * (longueur + largeur);
        double aire = longueur * largeur;

        System.out.println("Périmètre : " + perimetre);
        System.out.println("Aire      : " + aire);
        sc.close();
    }
}
```

Exemple d'exécution

```
Longueur : 5.5
Largeur  : 3
Périmètre : 17.0
Aire      : 16.5
```

Commentaires

À retenir

✓ Des dimensions réelles se stockent dans des double et se lisent avec nextDouble().

✓ Formules : périmètre = 2 × (longueur + largeur) ; aire = longueur × largeur.

✓ Le résultat d'une opération sur des double est un double : c'est pourquoi 17.0 s'affiche avec « .0 ».

Erreurs fréquentes et conseils

! Attention à la langue du système ! Un Scanner utilise par défaut les conventions de la langue de l'ordinateur. Sur un système en français, le séparateur décimal est la virgule : taper 5.5 provoquerait une InputMismatchException. La ligne sc.useLocale(Locale.US) impose le point. Sans cette ligne, saisissez 5,5.

! Oubli de parenthèses : 2 * longueur + largeur donne 14.0 au lieu de 17.0 (pour 5.5 et 3).
! N'utilisez pas int pour les dimensions : vous perdriez la partie décimale.

Exercice 5 : Pair ou impair

Énoncé

Lisez un entier et affichez s'il est pair ou impair.

Solution (fichier Exercice05.java)

```
import java.util.Scanner;

public class Exercice05 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Entrez un entier : ");
        int n = sc.nextInt();

        if (n % 2 == 0) {
            System.out.println(n + " est pair.");
        } else {
            System.out.println(n + " est impair.");
        }
        sc.close();
    }
}
```

Exemples d'exécution

```
Entrez un entier : 7
7 est impair.
Entrez un entier : -3
-3 est impair.
```

Commentaires

À retenir

✓ Un entier est pair si le reste de sa division par 2 est nul : n % 2 == 0.
✓ Structure if / else : un seul des deux blocs est exécuté.
✓ On teste l'égalité avec == (deux signes égal) et non avec = (qui est l'affectation).

Erreurs fréquentes et conseils

! Ne testez pas « impair » avec n % 2 == 1 : en Java, -3 % 2 vaut -1, pas 1. Le test d'imparité échouerait pour tous les nombres négatifs. Testez toujours le cas pair (== 0) et laissez l'autre cas dans le else.
! Écrire if (n % 2 = 0) est une erreur de compilation : = n'est pas ==.

Exercice 6 : Maximum de trois nombres

Énoncé

Lisez trois entiers et affichez le plus grand, sans utiliser Math.max.

Solution (fichier Exercice06.java)

```
import java.util.Scanner;

public class Exercice06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Premier entier : ");
        int a = sc.nextInt();
        System.out.print("Deuxième entier : ");
        int b = sc.nextInt();
        System.out.print("Troisième entier : ");
        int c = sc.nextInt();

        int max = a;          // on suppose que a est le plus grand
        if (b > max) {
            max = b;
        }
        if (c > max) {
            max = c;
        }

        System.out.println("Le plus grand est : " + max);
        sc.close();
    }
}
```

Exemple d'exécution

```
Premier entier : 14
Deuxième entier : 27
Troisième entier : 9
Le plus grand est : 27
```

Commentaires

À retenir

✓ Technique du « maximum courant » : on suppose que a est le plus grand, puis on le remplace si on rencontre mieux. C'est exactement la logique que vous réutiliserez pour un tableau (exercice 13).

✓ Deux if successifs suffisent : l'un compare b à max, l'autre compare c à max.

✓ Cette solution gère naturellement les égalités (par exemple 5, 5, 3).

Erreurs fréquentes et conseils

! N'initialisez jamais max à 0 : si les trois nombres sont négatifs, vous afficheriez 0, qui n'est pas l'un d'eux.

! Écrire une cascade « if (a > b && a > c) ... else if ... » fonctionne mais oublie facilement les cas d'égalité. La méthode du maximum courant est plus simple et plus sûre.

! Pour aller plus loin : l'opérateur ternaire permet d'écrire max = (b > max) ? b : max;

Exercice 7 : Mention

Énoncé

Lisez une note sur 20 et affichez la mention :

- moins de 10 : « Ajourné »
- de 10 à moins de 12 : « Passable »
- de 12 à moins de 14 : « Assez bien »
- de 14 à moins de 16 : « Bien »
- 16 et plus : « Très bien »

Affichez « Note invalide » si la note n'est pas comprise entre 0 et 20.

Solution (fichier Exercice07.java)

```
import java.util.Locale;
import java.util.Scanner;

public class Exercice07 {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        sc.useLocale(Locale.US);

        System.out.print("Note sur 20 : ");
        double note = sc.nextDouble();

        if (note < 0 || note > 20) {
            System.out.println("Note invalide");
        } else if (note < 10) {
            System.out.println("Ajourné");
        } else if (note < 12) {
            System.out.println("Passable");
        } else if (note < 14) {
            System.out.println("Assez bien");
        } else if (note < 16) {
            System.out.println("Bien");
        } else {
            System.out.println("Très bien");
        }
        sc.close();
    }
}
```

Exemples d'exécution

```
Note sur 20 : 13.5
Assez bien
Note sur 20 : 25
Note invalide
```

Commentaires

À retenir

- ✓ Une cascade if / else if / else n'exécute que la première branche dont la condition est vraie. Le test de validité vient en premier : on élimine les cas impossibles avant de classer.
- ✓ Dans chaque else if, on sait déjà que les conditions précédentes sont fausses. Il suffit donc d'écrire note < 12 (et non note >= 10 && note < 12), car on sait que note >= 10.
- ✓ Le type double permet des notes comme 13.5.

Erreurs fréquentes et conseils

- ! Sans le test de validité, la note 25 afficherait « Très bien ».
- ! Écrire plusieurs if indépendants au lieu de else if : une note de 15 afficherait plusieurs mentions.
- ! La notation mathématique 10 <= note < 12 n'existe pas en Java. Il faut combiner deux comparaisons avec && (et) ou || (ou).
- ! L'ordre des tests compte : si vous testez note < 16 avant note < 12, la mention « Assez bien » ne sera jamais affichée.

Exercice 8 : Table de multiplication

Énoncé

Lisez un entier n et affichez sa table de multiplication de 1 à 10.

```
7 x 1 = 7
7 x 2 = 14
...
```

Solution (fichier Exercice08.java)

```
import java.util.Scanner;

public class Exercice08 {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Entrez un entier n : ");
        int n = sc.nextInt();

        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }
        sc.close();
    }
}
```

Exemple d'exécution

```
Entrez un entier n : 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Commentaires

À retenir

- ✓ La boucle for (int i = 1; i <= 10; i++) se lit : « départ i = 1, tant que i <= 10, ajouter 1 à i après chaque tour ». On l'utilise quand le nombre d'itérations est connu à l'avance.
- ✓ On construit chaque ligne par concaténation : n + " x " + i + " = " + (n * i).
- ✓ La variable i n'existe qu'à l'intérieur de la boucle.

Erreurs fréquentes et conseils

- ! Erreur « off-by-one » : écrire i < 10 affiche la table de 1 à 9 seulement.
- ! Oublier i++ crée une boucle infinie (interrompez le programme avec Ctrl + C).
- ! Les parenthèses autour de n * i sont facultatives ici, car * est prioritaire sur +. Mais avec une addition (n + i), elles seraient indispensables : sans elles, Java concaténerait les deux nombres.

Exercice 9 : Somme des entiers

Énoncé

Calculez la somme des entiers de 1 à n (où n est lu au clavier), d'abord avec for, puis avec while.

Solution (fichier Exercice09.java)

```
import java.util.Scanner;

public class Exercice09 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Entrez un entier n : ");
        int n = sc.nextInt();

        // Version 1 : boucle for
        int sommeFor = 0;
        for (int i = 1; i <= n; i++) {
```

```
            sommeFor += i;
        }
        System.out.println("Somme (for)   : " + sommeFor);

        // Version 2 : boucle while
        int sommeWhile = 0;
        int i = 1;
        while (i <= n) {
            sommeWhile += i;
            i++;
        }
        System.out.println("Somme (while) : " + sommeWhile);

        // Vérification avec la formule n(n+1)/2
        System.out.println("Formule       : " + (n * (n + 1) / 2));
        sc.close();
    }
}
```

Exemple d'exécution

```
Entrez un entier n : 10
Somme (for)   : 55
Somme (while) : 55
Formule       : 55
```

Commentaires

À retenir

- ✓ Principe de l'accumulateur : une variable somme initialisée à 0, à laquelle on ajoute chaque valeur (somme += i).
- ✓ Avec while, c'est à vous d'initialiser le compteur avant la boucle et de l'incrémenter dans la boucle. Avec for, tout est regroupé dans l'en-tête.
- ✓ La formule de Gauss n(n + 1) / 2 sert de vérification : pour n = 10, on retrouve 55.
- ✓ La variable i du for n'existe que dans cette boucle, ce qui permet de redéclarer int i pour le while sans conflit.

Erreurs fréquentes et conseils

- ! Oublier i++ dans le while provoque une boucle infinie.
- ! Initialiser somme à 1 ou à n fausse le résultat.
- ! Pour de très grands n, un int déborde : la somme dépasse la limite des int vers n ≈ 65 535, et n * (n + 1) dans la formule dès n ≈ 46 340. Dans ce cas, utilisez long.

Exercice 10 : Factorielle

Énoncé

Calculez n! (par exemple 5! = 120). Que se passe-t-il avec n = 25 en int ? Essayez avec long.

Solution (fichier Exercice10.java)

```
import java.util.Scanner;

public class Exercice10 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Entrez un entier n : ");
        int n = sc.nextInt();

        // Version int
        int factInt = 1;
        for (int i = 2; i <= n; i++) {
```

```
            factInt *= i;
        }
        System.out.println(n + "! avec int  = " + factInt);

        // Version long
        long factLong = 1;
        for (int i = 2; i <= n; i++) {
            factLong *= i;
        }
        System.out.println(n + "! avec long = " + factLong);
        sc.close();
    }
}
```

Exemples d'exécution

```
Entrez un entier n : 5
5! avec int  = 120
5! avec long = 120
Entrez un entier n : 25
25! avec int  = 2076180480
25! avec long = 7034535277573963776
```

Commentaires

À retenir

- ✓ n! = 1 × 2 × … × n. L'accumulateur de produit démarre à 1 (élément neutre de la multiplication) et non à 0.
- ✓ Réponse à la question : avec int, 25! donne 2076180480, un résultat faux, et Java n'affiche aucune erreur. C'est un débordement (overflow) silencieux : un int ne dépasse pas 2 147 483 647, or 13! = 6 227 020 800 le dépasse déjà.
- ✓ Avec long, la limite est repoussée jusqu'à 20! (2 432 902 008 176 640 000). Mais 25! vaut en réalité 15 511 210 043 330 985 984 000 000 : le résultat 7034535277573963776 est donc lui aussi faux.
- ✓ Retenez : int est fiable jusqu'à 12!, long jusqu'à 20!.

Erreurs fréquentes et conseils

- ! Un programme qui compile et s'exécute sans erreur peut quand même donner un résultat faux. Vérifiez toujours les ordres de grandeur.
- ! Initialiser le produit à 0 donne toujours 0.
- ! Pour aller plus loin : la classe java.math.BigInteger permet de calculer des factorielles de taille quelconque, avec la méthode multiply().

Exercice 11 : Triangle d'étoiles

Énoncé

Lisez un entier n et affichez un triangle. Pour n = 4 :

```
*
**
***
****
```

Solution (fichier Exercice11.java)

```
import java.util.Scanner;

public class Exercice11 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Entrez un entier n : ");
        int n = sc.nextInt();
```

```
        for (int ligne = 1; ligne <= n; ligne++) {
            for (int j = 1; j <= ligne; j++) {
                System.out.print("*");
            }
            System.out.println();   // retour à la ligne
        }
        sc.close();
    }
}
```

Exemple d'exécution

```
Entrez un entier n : 4
*
**
***
****
```

Commentaires

À retenir

- ✓ Boucles imbriquées : la boucle externe parcourt les lignes (1 à n), la boucle interne affiche autant d'étoiles que le numéro de la ligne.
- ✓ print affiche sans passer à la ligne ; println() seul, après la boucle interne, termine la ligne.
- ✓ À chaque tour de la boucle externe, la boucle interne repart de zéro : c'est ce qui fait grandir le triangle.

Erreurs fréquentes et conseils

- ! Utiliser println dans la boucle interne : on obtient une étoile par ligne.
- ! Oublier le println() après la boucle interne : toutes les étoiles s'alignent sur une seule ligne.
- ! Pour aller plus loin : essayez un triangle inversé, ou aligné à droite (en affichant d'abord des espaces).

Exercice 12 : Nombre premier

Énoncé

Lisez un entier et dites s'il est premier ou non.

Solution (fichier Exercice12.java)

```
import java.util.Scanner;

public class Exercice12 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Entrez un entier : ");
        int n = sc.nextInt();

        boolean premier = true;
        if (n < 2) {
            premier = false;
        } else {
            for (int i = 2; i * i <= n; i++) {
                if (n % i == 0) {
                    premier = false;
                    break;          // inutile de continuer
                }
            }
        }

        if (premier) {
```

```
            System.out.println(n + " est premier.");
        } else {
            System.out.println(n + " n'est pas premier.");
        }
        sc.close();
    }
}
```

Exemples d'exécution

```
Entrez un entier : 29
29 est premier.
Entrez un entier : 12
12 n'est pas premier.
Entrez un entier : 1
1 n'est pas premier.
```

Commentaires

À retenir

✓ Un nombre est premier s'il est supérieur ou égal à 2 et n'a aucun diviseur autre que 1 et lui-même. On cherche un diviseur entre 2 et √n.

✓ La condition i * i <= n équivaut à i <= √n, sans recourir à Math.sqrt. Si n a un diviseur supérieur à √n, il en a aussi un plus petit : inutile de chercher plus loin.

✓ Le booléen premier joue le rôle de « drapeau » : on le suppose vrai, et on le passe à faux dès qu'un diviseur est trouvé. L'instruction break arrête alors la boucle.

Erreurs fréquentes et conseils

! 1 n'est pas premier. 0 et les nombres négatifs non plus : c'est pourquoi on traite n < 2 à part.

! Tester tous les diviseurs jusqu'à n - 1 donne le bon résultat, mais devient très lent pour de grands nombres.

! Ne mettez pas le message « est premier » à l'intérieur de la boucle : il serait affiché plusieurs fois. Concluez après la boucle.

Exercice 13 : Statistiques

Énoncé

Déclarez un tableau de 6 entiers, par exemple {12, 5, 8, 20, 3, 15}. Affichez tous les éléments, leur somme, leur moyenne, le minimum et le maximum.

Solution (fichier Exercice13.java)

```
public class Exercice13 {
    public static void main(String[] args) {
        int[] tab = {12, 5, 8, 20, 3, 15};

        int somme = 0;
        int min = tab[0];
        int max = tab[0];

        System.out.print("Éléments : ");
        for (int i = 0; i < tab.length; i++) {
            System.out.print(tab[i] + " ");
            somme += tab[i];
            if (tab[i] < min) {
                min = tab[i];
            }
            if (tab[i] > max) {
                max = tab[i];
            }
        }
        System.out.println();
```

```
        double moyenne = (double) somme / tab.length;

        System.out.println("Somme   : " + somme);
        System.out.println("Moyenne : " + moyenne);
        System.out.println("Minimum : " + min);
        System.out.println("Maximum : " + max);
    }
}
```

Exemple d'exécution

```
Éléments : 12 5 8 20 3 15
Somme   : 63
Moyenne : 10.5
Minimum : 3
Maximum : 20
```

Commentaires

À retenir

- ✓ tab.length donne la taille du tableau (sans parenthèses : c'est un attribut). Les indices vont de 0 à tab.length - 1.
- ✓ Un seul parcours suffit pour tout calculer : affichage, somme, minimum et maximum.
- ✓ Le minimum et le maximum sont initialisés avec tab[0], une valeur qui appartient au tableau. Ils sont ensuite mis à jour comme dans l'exercice 6.
- ✓ La moyenne s'écrit (double) somme / tab.length : la conversion en double est faite avant la division.

Erreurs fréquentes et conseils

- ! Sans le cast, somme / tab.length vaut 63 / 6 = 10 (division entière) et la moyenne 10.5 est perdue.
- ! Écrire i <= tab.length dans la boucle provoque une ArrayIndexOutOfBoundsException : l'indice 6 n'existe pas.
- ! Initialiser min à 0 donnerait un minimum faux (0 n'est pas dans le tableau).
- ! Variante : la boucle for (int v : tab) parcourt directement les valeurs, sans indice.

Exercice 14 : Recherche

Énoncé

Lisez un entier et dites s'il est présent dans le tableau précédent. Si oui, affichez sa position.

Solution (fichier Exercice14.java)

```
import java.util.Scanner;

public class Exercice14 {
    public static void main(String[] args) {
        int[] tab = {12, 5, 8, 20, 3, 15};
        Scanner sc = new Scanner(System.in);

        System.out.print("Valeur à chercher : ");
        int valeur = sc.nextInt();

        int position = -1;                  // -1 = pas encore trouvé
        for (int i = 0; i < tab.length; i++) {
            if (tab[i] == valeur) {
                position = i;
                break;
            }
        }

        if (position != -1) {
            System.out.println(valeur + " est présent à l'indice " + position + ".");
```

```
        } else {
            System.out.println(valeur + " n'est pas dans le tableau.");
        }
        sc.close();
    }
}
```

Exemples d'exécution

```
Valeur à chercher : 20
20 est présent à l'indice 3.
Valeur à chercher : 12
12 est présent à l'indice 0.
```

Commentaires

À retenir

✓ Recherche séquentielle : on parcourt le tableau jusqu'à trouver la valeur.

✓ La variable position vaut -1 au départ. C'est une valeur « sentinelle » : un indice valide est toujours ≥ 0, donc -1 signifie « pas trouvé ».

✓ L'affichage du résultat est fait après la boucle, en testant position != -1. L'instruction break évite de continuer inutilement.

Erreurs fréquentes et conseils

! Mettre le message « n'est pas dans le tableau » dans le for : il serait affiché à chaque élément différent.

! L'indice commence à 0 : 12 est à l'indice 0, c'est-à-dire en première place. Si vous voulez une position « humaine », affichez position + 1.

! En cas de doublons, ce programme renvoie seulement la première occurrence.

Exercice 15 : Inverser un tableau

Énoncé

Affichez le tableau à l'envers, puis inversez-le réellement sur place, sans créer de second tableau.

Solution (fichier Exercice15.java)

```
public class Exercice15 {
    public static void main(String[] args) {
        int[] tab = {12, 5, 8, 20, 3, 15};

        // 1) Affichage à l'envers (le tableau n'est pas modifié)
        System.out.print("À l'envers : ");
        for (int i = tab.length - 1; i >= 0; i--) {
            System.out.print(tab[i] + " ");
        }
        System.out.println();

        // 2) Inversion réelle sur place : on échange les extrémités
        for (int i = 0; i < tab.length / 2; i++) {
            int temp = tab[i];
            tab[i] = tab[tab.length - 1 - i];
            tab[tab.length - 1 - i] = temp;
        }

        System.out.print("Tableau inversé : ");
        for (int i = 0; i < tab.length; i++) {
            System.out.print(tab[i] + " ");
        }
        System.out.println();
    }
}
```

Exemple d'exécution

```
À l'envers : 15 3 20 8 5 12
Tableau inversé : 15 3 20 8 5 12
```

Commentaires

À retenir

✓ Affichage à l'envers : une boucle décroissante de tab.length - 1 jusqu'à 0. Le tableau n'est pas modifié.

✓ Inversion sur place : on échange tab[i] avec son symétrique tab[tab.length - 1 - i], à l'aide d'une variable temporaire temp.

✓ La boucle s'arrête à tab.length / 2 : chaque échange traite deux éléments à la fois.

✓ Les deux lignes affichées sont identiques, ce qui est normal : le tableau inversé se lit comme le tableau d'origine lu à l'envers.

Erreurs fréquentes et conseils

! Boucler jusqu'à tab.length : chaque paire serait échangée deux fois, et on retomberait sur le tableau d'origine.

! Oublier temp : avec tab[i] = tab[j]; tab[j] = tab[i];, la première valeur est écrasée, et les deux cases contiennent la même valeur.

! Le milieu d'un tableau de taille impaire ne bouge pas : c'est pourquoi la division entière length / 2 convient.

Exercice 16 : Mes premières méthodes

Énoncé

Écrivez les méthodes suivantes et testez-les dans main :

- int carre(int x) : renvoie x^2
- boolean estPair(int x)
- int maximum(int a, int b)
- double moyenne(int[] tab)

Solution (fichier Exercice16.java)

```
public class Exercice16 {

    static int carre(int x) {
        return x * x;
    }

    static boolean estPair(int x) {
        return x % 2 == 0;
    }

    static int maximum(int a, int b) {
        if (a > b) {
            return a;
        }
        return b;
    }

    static double moyenne(int[] tab) {
        int somme = 0;
        for (int v : tab) {
            somme += v;
        }
        return (double) somme / tab.length;
    }

    public static void main(String[] args) {
        System.out.println("carre(7)        = " + carre(7));
```

```
        System.out.println("estPair(10)     = " + estPair(10));
        System.out.println("estPair(7)      = " + estPair(7));
        System.out.println("maximum(8, 13)  = " + maximum(8, 13));

        int[] notes = {12, 5, 8, 20, 3, 15};
        System.out.println("moyenne(notes)  = " + moyenne(notes));
    }
}
```

Exemple d'exécution

```
carre(7)        = 49
estPair(10)     = true
estPair(7)      = false
maximum(8, 13)  = 13
moyenne(notes)  = 10.5
```

Commentaires

À retenir

- ✓ Une méthode se décrit par sa signature : type de retour, nom, paramètres. L'instruction return renvoie le résultat à l'appelant et termine la méthode.
- ✓ Le mot-clé static est nécessaire ici car main est static : une méthode static ne peut appeler que d'autres méthodes static de la même classe.
- ✓ Les méthodes se placent dans la classe, à côté de main, jamais à l'intérieur.
- ✓ Un tableau passé en paramètre n'est pas copié : la méthode travaille sur le tableau d'origine.

Erreurs fréquentes et conseils

- ! Oublier static provoque l'erreur « non-static method cannot be referenced from a static context ».
- ! Une méthode qui n'est pas void doit renvoyer une valeur dans tous les cas, sinon le compilateur signale « missing return statement ».
- ! Pour estPair, la ligne return x % 2 == 0; suffit. Écrire if (...) return true; else return false; est correct mais inutilement long.
- ! Dans moyenne, pensez au cast (double) pour éviter la division entière.

Exercice 17 : Conversion de température

Énoncé

Écrivez double celsiusVersFahrenheit(double c) (formule : c * 9 / 5 + 32), puis affichez un tableau de conversion de 0 à 100 °C par pas de 10.

Solution (fichier Exercice17.java)

```
public class Exercice17 {

    static double celsiusVersFahrenheit(double c) {
        return c * 9 / 5 + 32;
    }

    public static void main(String[] args) {
        System.out.println(" Celsius | Fahrenheit");
        System.out.println("---------+-----------");
        for (int c = 0; c <= 100; c += 10) {
            double f = celsiusVersFahrenheit(c);
            System.out.printf("  %5d  |  %7.1f%n", c, f);
        }
    }
}
```

Exemple d'exécution

```
 Celsius | Fahrenheit
---------+-----------
       0 |       32.0
      10 |       50.0
      20 |       68.0
      30 |       86.0
      40 |      104.0
      50 |      122.0
      60 |      140.0
      70 |      158.0
      80 |      176.0
      90 |      194.0
     100 |      212.0
```

Commentaires

À retenir

✓ On isole la formule dans une méthode réutilisable : le main se contente de l'appeler à chaque tour de boucle.

✓ Le pas de 10 s'obtient avec c += 10 dans l'en-tête du for.

✓ Passer un int à une méthode qui attend un double est permis : Java convertit automatiquement.

✓ printf permet de mettre en forme : %5d = entier sur 5 caractères, %7.1f = réel sur 7 caractères avec 1 décimale, %n = retour à la ligne.

Erreurs fréquentes et conseils

! Dans c * 9 / 5, c est un double donc la division est décimale. Mais si vous écriviez 9 / 5 * c, Java calculerait d'abord 9 / 5 = 1 (division entière) et le résultat serait faux.

! Évitez de piloter une boucle avec un compteur double (par exemple c += 0.1) : les erreurs d'arrondi peuvent faire sauter la dernière valeur. Utilisez un compteur entier.

! Sur un système en français, printf affiche la virgule décimale (32,0 au lieu de 32.0).

Exercice 18 : Analyse d'un mot

Énoncé

Lisez un mot et affichez sa longueur, sa première lettre, sa dernière lettre, et le mot en majuscules.

Solution (fichier Exercice18.java)

```java
import java.util.Scanner;

public class Exercice18 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Entrez un mot : ");
        String mot = sc.next();

        System.out.println("Longueur        : " + mot.length());
        System.out.println("Première lettre : " + mot.charAt(0));
        System.out.println("Dernière lettre : " + mot.charAt(mot.length() - 1));
        System.out.println("En majuscules   : " + mot.toUpperCase());
        sc.close();
    }
}
```

Exemple d'exécution

```
Entrez un mot : Programmation
Longueur        : 13
Première lettre : P
```

```
Dernière lettre : n
En majuscules   : PROGRAMMATION
```

Commentaires

À retenir

- ✓ length() donne le nombre de caractères ; charAt(i) donne le caractère à la position i (la numérotation commence à 0).
- ✓ La dernière lettre est donc à l'indice mot.length() - 1.
- ✓ next() lit un seul mot (jusqu'au premier espace), alors que nextLine() lit toute la ligne.
- ✓ Un String est immuable : toUpperCase() ne modifie pas mot, elle renvoie une nouvelle chaîne.

Erreurs fréquentes et conseils

- ! mot.charAt(mot.length()) provoque une StringIndexOutOfBoundsException : le dernier indice est length() - 1.
- ! Pour un String, length est une méthode et s'écrit avec des parenthèses : length(). Pour un tableau, c'est un attribut, sans parenthèses : tab.length. C'est une confusion très fréquente.
- ! Écrire mot.toUpperCase(); seul sur une ligne ne change rien : il faut utiliser ou afficher le résultat.

Exercice 19 : Compter les voyelles

Énoncé

Lisez une phrase et comptez le nombre de voyelles (a, e, i, o, u, y).

Solution (fichier Exercice19.java)

```
import java.util.Scanner;

public class Exercice19 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Entrez une phrase : ");
        String phrase = sc.nextLine().toLowerCase();

        int compteur = 0;
        for (int i = 0; i < phrase.length(); i++) {
            char c = phrase.charAt(i);
            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' || c == 'y') {
                compteur++;
            }
        }

        System.out.println("Nombre de voyelles : " + compteur);
        sc.close();
    }
}
```

Exemple d'exécution

```
Entrez une phrase : Bonjour tout le monde
Nombre de voyelles : 8
```

Commentaires

À retenir

- ✓ On parcourt la phrase caractère par caractère avec charAt(i), et on incrémente un compteur pour chaque voyelle.
- ✓ toLowerCase() est appliqué une fois au départ : on n'a pas besoin de tester les majuscules séparément.
- ✓ nextLine() est nécessaire ici pour lire une phrase contenant des espaces.
- ✓ Alternative plus concise : if ("aeiouy".indexOf(c) != -1) compteur++;

Erreurs fréquentes et conseils

! Un char se note entre apostrophes ('a') ; un String entre guillemets ("a"). Comparer un char à un String est une erreur de type.

! Les lettres accentuées (é, è, à, ô...) ne sont pas comptées par ce programme. Pour les compter, ajoutez-les à la liste des voyelles testées.

! Vérification manuelle de l'exemple : Bonjour (3) + tout (2) + le (1) + monde (2) = 8.

Exercice 20 : Palindrome

Énoncé

Lisez un mot et dites s'il s'agit d'un palindrome (radar, kayak, elle...).

Solution (fichier Exercice20.java)

```
import java.util.Scanner;

public class Exercice20 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Entrez un mot : ");
        String mot = sc.next().toLowerCase();

        boolean palindrome = true;
        int debut = 0;
        int fin = mot.length() - 1;

        while (debut < fin) {
            if (mot.charAt(debut) != mot.charAt(fin)) {
                palindrome = false;
                break;
            }
            debut++;
            fin--;
        }

        if (palindrome) {
            System.out.println(mot + " est un palindrome.");
        } else {
            System.out.println(mot + " n'est pas un palindrome.");
        }
        sc.close();
    }
}
```

Exemples d'exécution

```
Entrez un mot : radar
radar est un palindrome.
Entrez un mot : maison
maison n'est pas un palindrome.
```

Commentaires

À retenir

✓ Un palindrome se lit de la même façon dans les deux sens. On compare la première et la dernière lettre, puis la deuxième et l'avant-dernière, etc.

✓ Deux indices, debut et fin, avancent l'un vers l'autre. On s'arrête quand ils se croisent, ou dès qu'une paire diffère (break).

✓ toLowerCase() permet de traiter « Radar » comme « radar ».

✓ Alternative : new StringBuilder(mot).reverse().toString().equals(mot) renvoie directement la réponse.

Erreurs fréquentes et conseils

! Les char se comparent avec == ou != (ce sont des types primitifs). Mais pour comparer deux String, il faut toujours utiliser equals(), jamais == (qui compare les références, pas le contenu).

! Boucler jusqu'à mot.length() fonctionne mais refait chaque comparaison deux fois : il suffit d'aller jusqu'au milieu.

! Testez aussi avec kayak et elle : ce sont des palindromes.

Exercice 21 : La classe Étudiant

Énoncé

Créez une classe Etudiant avec :

- les attributs nom, prenom, note
- un constructeur
- une méthode afficher() qui affiche les informations
- une méthode estAdmis() qui renvoie true si la note est supérieure ou égale à 10

Dans main, créez trois étudiants et affichez ceux qui sont admis.

Solution (fichier Exercice21.java)

```
public class Exercice21 {
    public static void main(String[] args) {
        Etudiant e1 = new Etudiant("Benali", "Amina", 14.5);
        Etudiant e2 = new Etudiant("Alaoui", "Youssef", 8);
        Etudiant e3 = new Etudiant("Idrissi", "Salma", 10);

        System.out.println("Étudiants admis :");
        if (e1.estAdmis()) {
            e1.afficher();
        }
        if (e2.estAdmis()) {
            e2.afficher();
        }
        if (e3.estAdmis()) {
            e3.afficher();
        }
    }
}

class Etudiant {
    private String nom;
    private String prenom;
    private double note;

    public Etudiant(String nom, String prenom, double note) {
        this.nom = nom;
        this.prenom = prenom;
        this.note = note;
    }

    public void afficher() {
        System.out.println(prenom + " " + nom + " - note : " + note);
    }

    public boolean estAdmis() {
        return note >= 10;
    }
}
```

Exemple d'exécution

```
Étudiants admis :
Amina Benali - note : 14.5
Salma Idrissi - note : 10.0
```

Commentaires

À retenir

- ✓ Une classe est un modèle : elle décrit les attributs (les données) et les méthodes (les comportements). Chaque objet créé avec new en est une instance, avec ses propres valeurs.
- ✓ Le constructeur porte le même nom que la classe et n'a pas de type de retour. Il initialise les attributs. this.nom = nom copie le paramètre nom dans l'attribut nom de l'objet.
- ✓ Les attributs sont private (encapsulation) : on y accède uniquement par les méthodes de la classe.
- ✓ estAdmis() renvoie directement le résultat de la comparaison note >= 10. Salma est admise avec exactement 10, grâce au >=.

Erreurs fréquentes et conseils

- ! Sans this, l'instruction nom = nom; assigne le paramètre à lui-même : l'attribut reste null.
- ! Si vous écrivez void devant le constructeur, ce n'est plus un constructeur mais une méthode ordinaire, et new Etudiant(...) ne compile plus.
- ! Un fichier ne peut contenir qu'une seule classe public, dont le nom est celui du fichier. Ici Etudiant est dans le même fichier que le main, pour que l'exercice reste autonome. En pratique, on crée un fichier Etudiant.java.
- ! Pour aller plus loin : rangez les étudiants dans un tableau Etudiant[] et utilisez un for-each : for (Etudiant e : groupe) { if (e.estAdmis()) e.afficher(); }

Exercice 22 : Compte bancaire

Énoncé

Créez une classe Compte avec un attribut solde. Ajoutez les méthodes deposer(double montant) et retirer(double montant). Le retrait doit être refusé si le solde est insuffisant. Testez avec quelques opérations.

Solution (fichier Exercice22.java)

```
public class Exercice22 {
    public static void main(String[] args) {
        Compte c = new Compte(100);
        c.deposer(50);
        c.retirer(30);
        c.retirer(500);
        c.deposer(-20);
        System.out.println("Solde final : " + c.getSolde());
    }
}

class Compte {
    private double solde;

    public Compte(double soldeInitial) {
        this.solde = soldeInitial;
    }

    public void deposer(double montant) {
        if (montant > 0) {
            solde += montant;
            System.out.println("Dépôt de " + montant + " effectué. Solde : " + solde);
        } else {
            System.out.println("Dépôt refusé : le montant doit être positif.");
```

```
        }
    }

    public void retirer(double montant) {
        if (montant <= 0) {
            System.out.println("Retrait refusé : le montant doit être positif.");
        } else if (montant > solde) {
            System.out.println("Retrait refusé : solde insuffisant (" + solde + ").");
        } else {
            solde -= montant;
            System.out.println("Retrait de " + montant + " effectué. Solde : " + solde);
        }
    }

    public double getSolde() {
        return solde;
    }
}
```

Exemple d'exécution

```
Dépôt de 50.0 effectué. Solde : 150.0
Retrait de 30.0 effectué. Solde : 120.0
Retrait refusé : solde insuffisant (120.0).
Dépôt refusé : le montant doit être positif.
Solde final : 120.0
```

Commentaires

À retenir

- ✓ L'attribut solde est private : personne ne peut le modifier directement. Les seules portes d'entrée sont deposer et retirer, qui vérifient la validité de l'opération avant de modifier le solde. C'est le principe de l'encapsulation.
- ✓ Un retrait est refusé si montant > solde. Retirer exactement le solde est autorisé (le compte tombe à 0).
- ✓ getSolde() permet de consulter le solde sans pouvoir le modifier.
- ✓ Le test final montre le comportement attendu : après le retrait refusé et le dépôt refusé, le solde est resté à 120.

Erreurs fréquentes et conseils

- ! Écrire montant >= solde pour refuser le retrait interdirait de vider le compte.
- ! Oublier de contrôler que le montant est positif : deposer(-20) retirerait de l'argent, et retirer(-50) en ajouterait.
- ! Dans un vrai logiciel bancaire, on n'utilise pas double pour l'argent (erreurs d'arrondi) mais BigDecimal.

Pour aller plus loin : faire renvoyer un boolean à retirer (true si l'opération a réussi), pour que le programme appelant sache ce qui s'est passé.